

The order of image processing steps is crucial because each step affects the data quality for the next.

1. Warp

Perspective warping is applied at the beginning of the operation. The purpose is to correct any skew or geometric distortion in the image. Since the source and destination points for the transformation are predefined, the operation does not rely on prior image enhancement. Therefore, its position in the pipeline is flexible. Applying it first ensures that all subsequent processing steps operate on a spatially corrected image. Also this step won't affect the accuracy in the classifier but is important for humans.

In order to find the accurate 4 points, i have wrote a program which is not included in the main.py

```
import cv2
import numpy as np

def get_points(img):
    clicked = []

    def on_click(event, x, y, flags, param):
        if event == cv2.EVENT_LBUTTONDOWN:
            clicked.append((x, y))
            cv2.circle(temp, (x, y), 5, (0, 255, 0), -1)
            cv2.imshow("Click 4 Points", temp)
            print(f"Clicked: {x}, {y}")

    temp = img.copy()
    cv2.imshow("Click 4 Points", img)
    cv2.setMouseCallback("Click 4 Points", on_click)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    return np.float32(clicked[:4])

img = cv2.imread("im001-snow.jpg")
pts = get_points(img)
print(pts)
```

2. Inpaint Missing Area

Inpainting is used to recover missing or corrupted regions within the image. It is crucial to apply inpainting early in the pipeline, before any enhancement such as sharpening or gamma correction. This avoids the risk of inpainting being based on artificially modified pixel values, which could result in unrealistic textures. We use the Navier-Stokes (INPAINT_NS) method because it performs well when filling in larger regions.

3. Sharpen

Sharpening is performed after inpainting to enhance edge definition and texture clarity. This step emphasizes the fine details, such as object boundaries, which may have been blurred during warping or inpainting. Unsharp masking is used. The equation is $\text{sharpened} = \text{image} * 2.5 - \text{blurred} * 1.5$, which boosting the original and subtracting a blurred version

This improves visual clarity and prepares the image for effective denoising.

Process	Accuracy
Not applied Sharpen	0.89
Sharpen before remove noise	0.93
Sharpen after remove noise	0.87

When sharpening is applied afterward, it may reintroduce or amplify remaining noise and artifacts. That will decrease the accuracy in the classifier.

4. Noise Reduction

After sharpening, noise reduction is essential to suppress amplified noise. Median filtering removes salt-and-pepper noise, while bilateral filtering smooths textures while preserving edges. This combination ensures the image is clean but still sharp.

medianBlur\ bilateralFilter	d=5, sigmaColor=20, sigmaSpace=20	d=9, sigmaColor=40, sigmaSpace=40	d=13, sigmaColor=60, sigmaSpace=60
3	0.94	0.93	0.93
5	0.9	0.89	0.88
7	0.86	0.83	0.82

Increasing the size of medianBlur or the strength of bilateralFilter will blur useful details, which let the accuracy decrease. I choose configuration with medianBlur=3 and bilateralFilter d=9, sigmaColor=40, sigmaSpace=40 because it provides a strong balance between classification accuracy and noise suppression.

5. Auto Gamma Correction

Gamma correction is applied based on the image's average brightness.

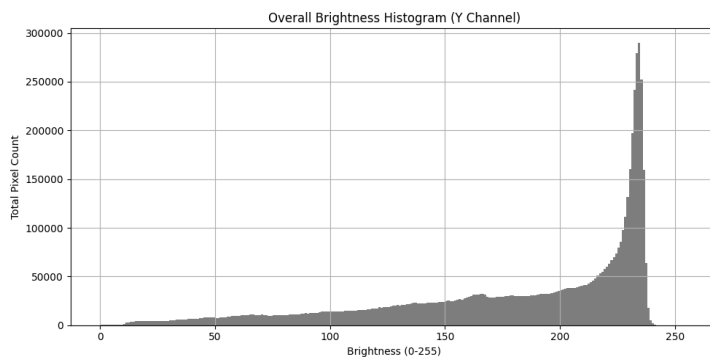
$\text{gamma value} = \text{current brightness} \setminus \text{target brightness}$

For the gamma value: (target brightness = 128)

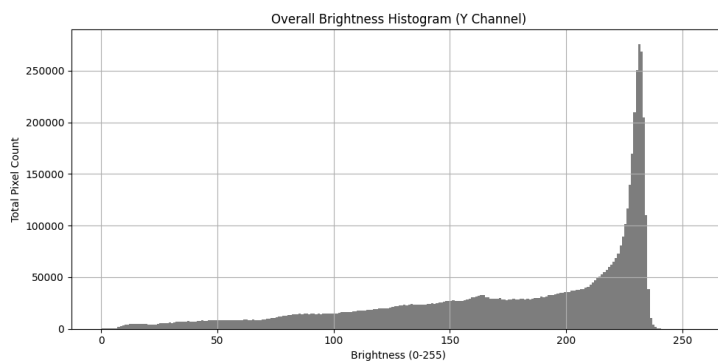
- If the image is too bright (brightness > 128), $\text{gamma} < 1 \rightarrow$ it darkens the image.
- If the image is too dark (brightness < 128), $\text{gamma} > 1 \rightarrow$ it brightens the image.

This enhances underexposed or overexposed images by adjusting luminance non-linearly. To avoid extreme changes, the gamma value is clipped between 0.9 and 1.5 to keep the image stable. Performing this step after denoising ensures noise is not unintentionally enhanced during brightening.

No gamma correction:



Have gamma correction:

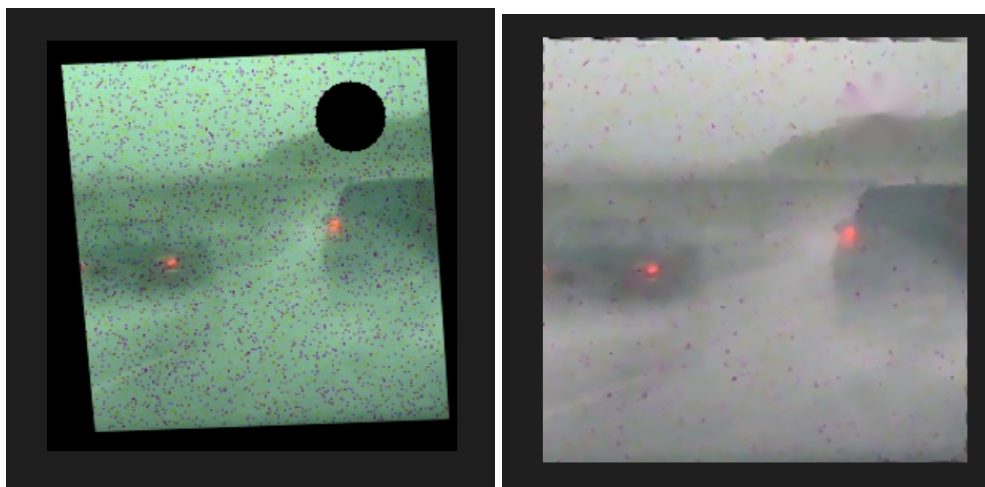


The histogram shows that after gamma correction, the pixel intensity distribution shifts closer to the target brightness value of 128, indicating more balanced exposure across the image.

6. Gray World White Balance

The final step adjusts color balance using the Gray World assumption. It scales each color channel to bring the average closer to neutral gray. Placing this at the end ensures accurate color correction after contrast and brightness adjustments.

Process	Accuracy
Not applied color balance	0.88
Applied color balance	0.93



The photo has a strong green tint., after applying color balance operation, indicating a better white balance.

Code quality

i have check the code of my program to make sure it follows the PEP8 coding style

```
PS C:\Users\computer\OneDrive\桌面\image-processing-files> python -m flake8 main.py
>>
PS C:\Users\computer\OneDrive\桌面\image-processing-files>
```